

Project: Smart Task Manager with AI-like UX

Goal

Build a **high-performance task management dashboard** with:

- Optimistic UI updates
 - Smart filtering/search (deferred rendering)
 - Stable IDs for accessibility
 - Custom reusable hooks
 - Efficient rendering using memoization
 - Modern React patterns (including experimental hooks)
-

Core Concept

Instead of building a basic TODO app (which is overdone), students will build:

👉 A **Smart Task Manager** that simulates:

- Real-time UX
 - Server interactions
 - Performance optimizations
 - Reusable architecture
-

Project Structure

```
src/
├── components/
│   ├── TaskList.jsx
│   ├── TaskItem.jsx
│   ├── TaskForm.jsx
│   ├── SearchBar.jsx
│   └── Filters.jsx
├── hooks/
│   ├── useTasks.js           (Custom Hook)
│   └── useDebounceSearch.js (Custom Hook)
├── utils/
│   └── api.js                 (fake async API)
├── App.jsx
└── main.jsx
```

✿ TASK BREAKDOWN (STEP-BY-STEP)

✿ Step 1: Project Setup & UI Skeleton

What to do:

- Create a React project (prefer Vite for speed)
- Design a basic layout:
 - Header (App Title)
 - Task Input Form
 - Search Bar
 - Filter Section (All / Completed / Pending)
 - Task List

Focus:

- Clean UI separation into components
- Think in component hierarchy

👉 Outcome:

A static UI with no logic yet

🧠 Step 2: Build a Custom Hook for Task Management

What to do:

- Create a custom hook to manage:
 - Task list
 - Loading state
 - Function to fetch tasks
 - Function to update tasks

Focus:

- Move all business logic OUT of components
- Keep components clean and reusable

👉 Outcome:

A reusable task management system (like a mini state manager)

🌐 Step 3: Simulate Backend (Async Behavior)

What to do:

- Create fake API functions:
 - Fetch tasks (with delay)
 - Add task (with delay)
- Connect your custom hook to these APIs

Focus:

- Handle loading states properly
- Think like you're working with a real backend

👉 Outcome:

App now behaves like a real-world product (not static)

```
JavaScript

// utils/api.js
export async function fetchTasks() {
  await new Promise(r => setTimeout(r, 1000));
  return [
    { id: 1, title: "Learn React", completed: false },
    { id: 2, title: "Build Project", completed: true }
  ];
}

export async function addTask(task) {
  await new Promise(r => setTimeout(r, 1000));
  return { ...task, id: Date.now() };
}
```

⚡ Step 4: Implement Optimistic UI (useOptimistic)

What to do:

- When user adds a task:
 - Show it instantly in UI

- Then sync with backend
- If backend fails → rollback changes

Focus:

- User experience over waiting time
- Handling temporary vs actual state

👉 Outcome:

Fast, responsive UI like modern SaaS apps

Step 5: Handle Form Submission (useActionState)

What to do:

- Manage form submission using an action-based pattern
- Track:
 - Pending state (loading)
 - Success response
 - Error handling

Focus:

- Avoid messy multiple useState calls
- Think in terms of state transitions

👉 Outcome:

Clean, scalable form handling logic

Step 6: Accessibility (useId)

What to do:

- Assign unique IDs to form inputs
- Link labels properly with inputs

Focus:

- Accessibility standards (important in real companies)
- Avoid ID conflicts in reusable components

👉 Outcome:

Accessible and production-ready forms

🔍 Step 7: Implement Search with Deferred Rendering

What to do:

- Add a search input
- Delay heavy filtering logic using deferred value

Focus:

- Prevent UI lag while typing
- Separate "input state" vs "processing state"

👉 Outcome:

Smooth typing experience even with large data

🧠 Step 8: Optimize Filtering (useMemo)

What to do:

- Filter tasks based on:
 - Search input
 - Task status (completed/pending)
- Memoize the filtered result

Focus:

- Avoid unnecessary recalculations
- Understand when optimization is needed

👉 Outcome:

Efficient rendering even with large task lists

💬 Step 9: Handle Stable Event Logic (useEffectEvent)

What to do:

- Track task updates inside effects

- Avoid stale closures when working with state inside effects

Focus:

- Clean effect logic
- Avoid bugs caused by outdated values

 **Outcome:**

More predictable and stable behavior

Step 10: Build a Custom Hook for Debouncing

What to do:

- Create a reusable hook that:
 - Delays value updates
 - Clears previous timers

Focus:

- Reusability
- Real-world UX improvements

 **Outcome:**

Reusable performance hook used across projects